

DOCUMENTACIÓN TÉCNICA

Rutina Preprogramada de Trayectoria Cuadrada

Control de locomoción para robot Unitree R1 EDU mediante Unitree R1 LocoClient

Campo	Detalle
Paquete / Script	r1_square_mode.cpp
Versión	Documento técnico v1.0
Plataforma	Linux / C++ / entorno de compilación compatible con Unitree SDK
Robot / Hardware	Unitree R1 EDU
SDK / Framework	Unitree R1 Loco API, r1_loco_client.hpp, ChannelFactory
Script base	Rutina C++ de locomoción por velocidad con LocoClient
Autor	Robotics 4.0
Fecha	30 de junio de 2026

0. Índice

1. Descripción General	3
1.1. Problema que resuelve	3
1.2. Entradas, procesamiento y salida	3
1.3. Componentes desarrollados	4
2. Arquitectura del Sistema	4
2.1. Descripción por capas	4
2.2. Comunicación e interfaces	5
2.3. Trayectoria ejecutada	6
3. Requisitos y Dependencias	6
3.1. Requisitos mínimos	6
3.2. Comandos de verificación	6
4. Organización de Archivos	7
4.1. Estructura recomendada	7
4.2. Archivos obligatorios	7
4.3. Nombres y rutas	8
5. Formato de Entrada	8
5.1. Campos de entrada	8
5.2. Ejemplo mínimo funcional	8
5.3. Validación de entrada	8
6. Código Fuente y Funcionamiento Interno	9
6.1. Importaciones	9
6.2. Inicialización de comunicación	9
6.3. Captura de parámetros	10
6.4. Cálculo temporal	10
6.5. Secuencia de activación	10
6.6. Margen de espera entre lados	11
6.7. Comandos de movimiento	11
6.8. Cierre de movimiento	11
6.9. Código fuente completo de referencia	12
7. Flujo de Ejecución	13
7.1. Preparación del entorno	13
7.2. Compilación sugerida	13
7.3. Ejecución	14
7.4. Verificación del resultado	14
8. Seguridad Operacional	15
8.1. Condiciones antes de ejecutar	15
8.2. Acciones no recomendadas	15
8.3. Detención segura	15
8.4. Interrupción con Ctrl+C	15
9. Guía de Uso Rápido	16

10. Problemas Conocidos y Soluciones	16
11. Extensión y Mantenimiento	17
11.1. Agregar pausa explícita entre lados	18
11.2. Agregar trayectoria con giro	18
11.3. Parametrizar la interfaz de red	18
11.4. Agregar límites máximos de seguridad	18
11.5. Partes que no deben modificarse sin precaución	19
11.6. Validación posterior a modificaciones	19
12. Resumen de Actividades	20
13. Conclusión Técnica	20



1. Descripción General

Esta documentación describe una rutina preprogramada en C++ para ejecutar una trayectoria cuadrada con el robot Unitree R1 EDU. El programa controla la locomoción mediante comandos temporizados de velocidad lineal enviados a través de `unitree::robot::r1::LocoClient`.

La lógica del sistema es directa: el usuario ingresa por terminal la longitud del lado del cuadrado y la velocidad de desplazamiento; el programa calcula el tiempo requerido para cada lado usando la relación física $\text{tiempo} = \text{distancia} / \text{velocidad}$; después activa el cliente de locomoción y ejecuta cuatro comandos secuenciales de velocidad: avance, desplazamiento lateral izquierdo, retroceso y desplazamiento lateral derecho. Al finalizar, envía una orden explícita de detención mediante `StopMove()`.

Objetivo operativo. Permitir que el robot R1 EDU ejecute una trayectoria cuadrada aproximada sin programar puntos de navegación ni depender de un sistema de odometría cerrada. La rutina es útil para pruebas iniciales de locomoción, validación de comunicación SDK, demostraciones controladas y verificación básica de respuesta en los ejes X/Y.

1.1

En pruebas de locomoción robótica es frecuente requerir rutinas simples, repetibles y parametrizables. Este código reemplaza un flujo manual donde el operador tendría que enviar comandos individuales para cada tramo del movimiento. La herramienta centraliza el cálculo temporal, la secuencia de comandos y la detención final en un único programa ejecutable.

1.2

Etapa	Descripción	Resultado
Entrada	El operador ingresa distancia del lado en metros y velocidad en metros por segundo.	Parámetros numéricos <code>distance</code> y <code>speed</code> .
Validación	El programa comprueba que distancia y velocidad sean mayores que cero.	Ejecución permitida o terminación con error.
Procesamiento	Calcula <code>duration = distance / speed</code> .	Tiempo de ejecución para cada lado del cuadrado.
Ejecución	Envía cuatro comandos <code>SetVelocity(vx, vy, wz, duration)</code> .	Desplazamiento secuencial en +X, +Y, -X y -Y.
Salida	Ejecuta <code>StopMove()</code> .	Robot detenido al finalizar la rutina.

1.3

Componente	Tipo	Descripción	Estado
<code>r1_square_mode.cpp</code>	Script C++	Programa principal que inicializa comunicación, recibe parámetros, calcula duración y ejecuta los cuatro lados del cuadrado.	Implementado
<code>LocoClient</code>	Cliente SDK	Interfaz de locomoción del R1 utilizada para iniciar, controlar velocidad y detener movimiento.	Requerido
<code>ChannelFactory</code>	Comunicación SDK	Inicializa el canal de red con el robot mediante una interfaz Ethernet específica.	Requerido
Terminal de usuario	Interfaz textual	Permite ingresar distancia y velocidad de ejecución.	Implementado
Control temporizado	Lógica interna	Calcula duración por lado y añade margen de espera de 500 ms entre comandos.	Implementado
<code>StopMove()</code>	Cierre operacional	Orden explícita de detención al finalizar la trayectoria.	Implementado

2. Arquitectura del Sistema

La herramienta opera sobre una arquitectura de control por comandos de velocidad. El computador que ejecuta el programa se comunica con el robot R1 EDU mediante una interfaz Ethernet configurada en el SDK de Unitree. El programa no crea un nodo ROS2 propio ni publica tópicos directamente; su canal operativo principal es el cliente de locomoción `LocoClient`.

2.1

Modelo de ejecución. La rutina es de lazo abierto: calcula cuánto tiempo debe mantenerse una velocidad, pero no corrige la trayectoria con odometría, percepción, localización ni realimentación de posición. Por tanto, el cuadrado real puede variar según superficie, fricción, respuesta dinámica del robot y latencia de comunicación.

Capa / Módulo	Función	Entrada	Salida
Operador	Define los parámetros físicos de la trayectoria.	Distancia y velocidad.	Valores capturados por <code>std::cin</code> .
Programa C++	Valida entradas, calcula duración y ejecuta la secuencia.	<code>distance, speed</code> .	Comandos <code>SetVelocity</code> .
Unitree SDK	Gestiona comunicación y abstracción de locomoción.	Llamadas a <code>LocoClient</code> .	Paquetes de control hacia el robot.
Red Ethernet	Transporta los mensajes entre computador y robot.	Interfaz definida en <code>network_interface</code> .	Comunicación física con R1 EDU.
Robot R1 EDU	Ejecuta la locomoción solicitada.	Velocidades <code>vx, vy, wz</code> .	Movimiento físico sobre el plano.

2.2

Canal / Interfaz	Uso	Entorno	Observación
<code>eth10</code>	Interfaz Ethernet declarada en el código.	Computador conectado al robot.	Debe cambiarse si la interfaz real tiene otro nombre.
<code>ChannelFactory::Instance(interface)</code>	Inicialización de comunicación Unitree.	SDK C++.	El parámetro 0 corresponde al modo de comunicación de red para robot físico.
<code>LocoClient::Init()</code>	Inicializa el cliente de locomoción.	Proceso C++.	Debe ejecutarse antes de enviar comandos.
<code>LocoClient::SetTimeout(Def)</code>	Define tiempo máximo de espera.	Proceso C++.	Permite detectar bloqueos o falta de respuesta del SDK.
<code>SetVelocity(vx, vy, wz, duration)</code>	Envía velocidad lineal/angular durante un intervalo.	Robot físico.	Controla movimiento en X/Y y yaw.
<code>StopMove()</code>	Detiene la locomoción.	Robot físico.	Debe ejecutarse al finalizar la rutina.

2.3

La rutina no gira el robot noventa grados en cada esquina. El robot conserva aproximadamente su orientación inicial y se desplaza de forma translacional sobre los ejes del marco de locomoción utilizado por el SDK.

Listing 1: Secuencia conceptual de movimiento

```
Inicio
-> Lado 1: vx = +speed, vy = 0,      wz = 0
-> Lado 2: vx = 0,      vy = +speed, wz = 0
-> Lado 3: vx = -speed, vy = 0,      wz = 0
-> Lado 4: vx = 0,      vy = -speed, wz = 0
Fin
-> StopMove()
```

3. Requisitos y Dependencias

La ejecución requiere un entorno capaz de compilar C++ y acceder a las cabeceras/-librerías del SDK de Unitree para el R1. Además, el computador debe tener comunicación de red funcional con el robot.

3.1

Requisito	Versión / Valor	Verificación
Sistema operativo	Linux compatible con SDK Unitree.	<code>uname -a</code>
Compilador C++	C++17 recomendado.	<code>g++ -version</code>
CMake	Recomendado si se integra al proyecto SDK.	<code>cmake -version</code>
SDK Unitree R1	Cabeceras <code>r1_loco_api.hpp</code> y <code>r1_loco_client.hpp</code> .	Verificar rutas <code>include/unitree/robot/r1/loco/</code> .
Red Ethernet	Interfaz conectada al robot, por defecto <code>eth10</code> .	<code>ip link show</code>
Robot	Unitree R1 EDU encendido y en estado seguro para locomoción.	Confirmación visual y estado operativo del robot.
Área de prueba	Superficie despejada, plana y con espacio suficiente.	Inspección física antes de ejecutar.

3.2

Listing 2: Verificación básica del entorno

```
uname -a
```

```
g++ --version
cmake --version
ip link show
ip addr show eth10
```

Si la interfaz `eth10` no existe, debe identificarse la interfaz real conectada al robot:

Listing 3: Búsqueda de interfaces de red disponibles

```
ip link show
ip addr
```

Condición crítica. No ejecutar la rutina si no se ha confirmado la interfaz de red correcta. Una interfaz incorrecta puede impedir el envío de comandos o dejar el programa esperando respuesta del SDK.

4. Organización de Archivos

La rutina puede integrarse en un proyecto existente del SDK Unitree o mantenerse como un ejemplo propio dentro del repositorio de trabajo de Robotics 4.0. La recomendación es conservar una carpeta específica para pruebas del R1 y nombrar el archivo de forma explícita.

4.1

Listing 4: Árbol sugerido de proyecto

```
r1_loco_routines/
|-- CMakeLists.txt
|-- src/
|   '-- r1_square_mode.cpp
|-- build/
'-- README.md
```

4.2

Archivo / Carpeta	Obligatorio	Función
<code>src/r1_square_mode.cpp</code>	Sí	Contiene la rutina principal de trayectoria cuadrada.
<code>CMakeLists.txt</code>	Sí, si se compila con CMake	Define el ejecutable y enlaza dependencias del SDK.
Cabeceras Unitree	Sí	Proveen las clases <code>LocoClient</code> y <code>ChannelFactory</code> .
<code>build/</code>	No inicial	Carpeta generada durante la compilación.
<code>README.md</code>	Opcional	Registro breve de uso interno.

4.3

El código no depende de archivos externos de configuración. Sin embargo, sí depende del nombre de interfaz definido en esta línea:

Listing 5: Interfaz de red definida en el código

```
std::string network_interface = "eth10";
```

Si el computador usa otro nombre de interfaz, esta variable debe modificarse antes de compilar. Se recomienda evitar nombres ambiguos y documentar en el README cuál interfaz corresponde a la conexión física con el robot.

5. Formato de Entrada

La herramienta no consume archivos JSON, TXT, YAML ni CSV. Su entrada es interactiva por terminal. El operador debe ingresar dos valores numéricos de tipo flotante.

5.1

Campo	Tipo	Unidad	Descripción
distance	float	metros	Longitud de cada lado del cuadrado. Debe ser mayor que cero.
speed	float	metros/segundo	Velocidad lineal aplicada en cada lado. Debe ser mayor que cero.

5.2

Listing 6: Entrada sugerida para una prueba conservadora

```
=== MODO CUADRADO ===
Ingresa la longitud del lado del cuadrado (en metros, ej. 1.0): 0.5
Ingresa la velocidad (m/s, ej. 0.2): 0.2
```

Con estos valores, el programa calcula:

$$\text{duration} = \frac{\text{distance}}{\text{speed}} = \frac{0,5 \text{ m}}{0,2 \text{ m/s}} = 2,5 \text{ s}$$

Cada lado se ejecutará durante aproximadamente 2.5 segundos, más el margen de espera interno de 0.5 segundos antes de pasar al siguiente comando.

5.3

El programa termina con error si cualquiera de los dos valores es menor o igual a cero:

Listing 7: Validación de parámetros físicos

```
if (speed <= 0 || distance <= 0) {
```

```
std::cerr << "Error: La velocidad y la distancia deben ser mayores a 0." << std::endl;
return 1;
}
```

6. Código Fuente y Funcionamiento Interno

Esta sección explica la lógica interna del código por bloques funcionales. La rutina se implementa dentro de la función `main()`, sin clases auxiliares propias ni archivos de configuración externos.

6.1

Listing 8: Librerías utilizadas

```
#include <chrono>
#include <iostream>
#include <thread>
#include <vector>

#include "unitree/robot/r1/loco/r1_loco_api.hpp"
#include "unitree/robot/r1/loco/r1_loco_client.hpp"
```

Elemento	Responsabilidad
chrono	Manejo de duraciones y pausas temporales.
iostream	Entrada y salida por terminal.
thread	Ejecución de pausas con <code>std::this_thread::sleep_for</code> .
vector	Incluida, pero no utilizada en esta versión. Puede eliminarse sin afectar la lógica actual.
r1_loco_api.hpp	API de locomoción del R1.
r1_loco_client.hpp	Cliente C++ para enviar comandos de locomoción.

6.2

Listing 9: Inicialización de red y cliente

```
std::string network_interface = "eth10";
unitree::robot::ChannelFactory::Instance()->Init(0, network_interface);

unitree::robot::r1::LocoClient client;
client.Init();
client.SetTimeout(10.f);
```

Elemento	Función	Riesgo si falla
network_interface	Define la interfaz Ethernet usada para comunicarse con el robot.	Sin comunicación con el R1.
ChannelFactory::Init	Inicializa la capa de comunicación del SDK.	El cliente no puede enviar comandos.
client.Init()	Prepara el cliente de locomoción.	Fallo de ejecución o comandos ignorados.
client.SetTimeout(10.f)	Establece tiempo máximo de espera de 10 segundos.	Bloqueos prolongados si hay problemas de red.

6.3

Listing 10: Entrada por terminal

```
float distance = 0.0f;
float speed = 0.2f;

std::cout << "=== MODO CUADRADO ===" << std::endl;
std::cout << "Ingresa la longitud del lado del cuadrado (en metros, ej. 1.0): ";
std::cin >> distance;

std::cout << "Ingresa la velocidad (m/s, ej. 0.2): ";
std::cin >> speed;
```

El valor por defecto de speed es 0.2f, pero se sobrescribe cuando el usuario ingresa una velocidad por terminal.

6.4

Listing 11: Cálculo de duración por lado

```
float duration = distance / speed;
```

El cálculo representa una aproximación cinemática simple:

$$t = \frac{d}{v}$$

Donde t es el tiempo por lado, d es la distancia del lado y v es la velocidad lineal. El programa no mide el desplazamiento real ni corrige errores acumulados.

6.5

Listing 12: Inicio seguro de secuencia

```
std::cout << "\nIniciando secuencia en 3 segundos. ¡Despeja el área!" << std::endl;
std::this_thread::sleep_for(std::chrono::seconds(3));

client.Start();
std::this_thread::sleep_for(std::chrono::seconds(1));
```

La pausa inicial de 3 segundos permite despejar el área. Luego, `client.Start()` activa el cliente de locomoción antes de enviar los comandos de velocidad.

6.6

Listing 13: Conversión de duración y margen de estabilización

```
int sleep_ms = static_cast<int>(duration * 1000) + 500;
```

El programa espera el tiempo de ejecución del movimiento más 500 ms. Este margen busca evitar transiciones inmediatas entre direcciones opuestas o perpendiculares.

6.7

Listing 14: Ejecución secuencial del cuadrado

```
client.SetVelocity(speed, 0.0f, 0.0f, duration); // +X
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

client.SetVelocity(0.0f, speed, 0.0f, duration); // +Y
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

client.SetVelocity(-speed, 0.0f, 0.0f, duration); // -X
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

client.SetVelocity(0.0f, -speed, 0.0f, duration); // -Y
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));
```

Paso	Movimiento	vx	vy	wz	Efecto
1	Adelante	+speed	0	0	Avance longitudinal.
2	Izquierda	0	+speed	0	Desplazamiento lateral.
3	Atrás	-speed	0	0	Retroceso longitudinal.
4	Derecha	0	-speed	0	Retorno lateral.

6.8

Listing 15: Detención final

```
std::cout << "Cuadrado completado. Deteniendo robot." << std::endl;
client.StopMove();
```

La función `StopMove()` es la acción de cierre más importante del programa. Sin esta llamada, el robot podría no recibir una orden explícita de finalización desde el proceso del usuario.

6.9

Listing 16: Archivo principal r1_square_mode.cpp

```
#include <chrono>
#include <iostream>
#include <thread>
#include <vector>

#include "unitree/robot/r1/loco/r1_loco_api.hpp"
#include "unitree/robot/r1/loco/r1_loco_client.hpp"

int main() {
    // 1. Inicialización de la red (Ajusta "eth10" a tu interfaz real si es necesario)
    std::string network_interface = "eth10";
    unitree::robot::ChannelFactory::Instance()->Init(0, network_interface);

    unitree::robot::r1::LocoClient client;
    client.Init();
    client.SetTimeout(10.f);

    // 2. Pedir dimensiones al usuario en la terminal
    float distance = 0.0f;
    float speed = 0.2f;

    std::cout << "=== MODO CUADRADO ===" << std::endl;
    std::cout << "Ingresa la longitud del lado del cuadrado (en metros, ej. 1.0): ";
    std::cin >> distance;

    std::cout << "Ingresa la velocidad (m/s, ej. 0.2): ";
    std::cin >> speed;

    if (speed <= 0 || distance <= 0) {
        std::cerr << "Error: La velocidad y la distancia deben ser mayores a 0." << std::endl;
        return 1;
    }

    // 3. Cálculo de la duración
    // Fórmula física básica: Tiempo = Distancia / Velocidad
    float duration = distance / speed;

    std::cout << "\nResumen: El robot recorrerá " << distance << "m a " << speed
        << "m/s." << std::endl;
    std::cout << "Tiempo por lado: " << duration << " segundos." << std::endl;

    std::cout << "\nIniciando secuencia en 3 segundos. ¡Despeja el área!" << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(3));

    client.Start(); // Asegura que el robot esté activo
    std::this_thread::sleep_for(std::chrono::seconds(1));

    // Margen de seguridad: Agregamos 0.5s de pausa entre movimientos para estabilizar
    int sleep_ms = static_cast<int>(duration * 1000) + 500;

    // 4. Secuencia del Cuadrado (Desplazamiento Translacional)

    // LADO 1: Hacia adelante (+X)
    std::cout << "[1/4] Moviendo hacia adelante..." << std::endl;
    client.SetVelocity(speed, 0.0f, 0.0f, duration);
    std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

    // LADO 2: Hacia la izquierda (+Y)
```

```
std::cout << "[2/4] Moviendo hacia la izquierda..." << std::endl;
client.SetVelocity(0.0f, speed, 0.0f, duration);
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

// LADO 3: Hacia atrás (-X)
std::cout << "[3/4] Moviendo hacia atrás..." << std::endl;
client.SetVelocity(-speed, 0.0f, 0.0f, duration);
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

// LADO 4: Hacia la derecha (-Y)
std::cout << "[4/4] Moviendo hacia la derecha..." << std::endl;
client.SetVelocity(0.0f, -speed, 0.0f, duration);
std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));

// 5. Finalizar
std::cout << "Cuadrado completado. Deteniendo robot." << std::endl;
client.StopMove();

return 0;
}
```

7. Flujo de Ejecución

El flujo de ejecución debe realizarse de forma controlada. Antes de correr el programa, se debe comprobar que el robot esté en un área segura, que la interfaz Ethernet sea correcta y que el binario haya sido compilado sin errores.

7.1

1. Conectar el computador al robot R1 EDU mediante Ethernet.
2. Verificar el nombre de la interfaz de red.
3. Ajustar `network_interface` en el código si no corresponde a `eth10`.
4. Confirmar que las cabeceras del SDK estén disponibles.
5. Compilar el ejecutable.

7.2

Si el código se integra en un proyecto CMake existente del SDK, crear o adaptar un `CMakeLists.txt`. La estructura exacta de enlazado puede variar según la instalación del SDK; el siguiente bloque muestra una base de referencia que debe ajustarse a las librerías disponibles del proyecto Unitree local.

Listing 17: CMakeLists.txt de referencia

```
cmake_minimum_required(VERSION 3.10)
project(r1_loco_routines)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(r1_square_mode src/r1_square_mode.cpp)
```

```
# Ajustar estas líneas según la estructura real del SDK Unitree instalado.
# target_include_directories(r1_square_mode PRIVATE /ruta/al/unitree_sdk/include)
# target_link_libraries(r1_square_mode PRIVATE unitree_sdk2)
```

Listing 18: Construcción del ejecutable

```
cd r1_loco_routines
mkdir -p build
cd build
cmake ..
make -j$(nproc)
```

7.3

Listing 19: Ejecución del modo cuadrado

```
cd r1_loco_routines/build
./r1_square_mode
```

Durante la ejecución, ingresar valores conservadores para la primera prueba:

Listing 20: Primera prueba recomendada

```
Distancia: 0.3
Velocidad: 0.1
```

Estos valores reducen el desplazamiento inicial y permiten verificar que los signos de los ejes X/Y correspondan al comportamiento esperado del robot.

7.4

Elemento a verificar	Comportamiento esperado	Acción si falla
Comunicación	El programa avanza después de <code>client.Start()</code> .	Revisar interfaz Ethernet y conexión física.
Lado 1	Movimiento hacia adelante.	Revisar convención de ejes o modo de locomoción.
Lado 2	Movimiento lateral hacia la izquierda.	Confirmar si el robot permite desplazamiento lateral en el modo activo.
Lado 3	Movimiento hacia atrás.	Reducir velocidad si hay inestabilidad.
Lado 4	Movimiento lateral hacia la derecha.	Verificar que no existan obstáculos en el retorno.
Final	El robot se detiene al ejecutar <code>StopMove()</code> .	Usar parada física si no se detiene.

8. Seguridad Operacional

Esta rutina controla movimiento físico del robot. Por tanto, debe tratarse como una ejecución de hardware real, no como una prueba puramente de software.

Advertencia crítica. Antes de ejecutar el programa, despejar completamente el área alrededor del robot. La trayectoria esperada es cuadrada, pero el movimiento real puede desviarse por fricción, latencia, deriva de orientación, saturaciones del controlador o errores de configuración de red.

8.1

- El robot debe estar encendido, estable y en modo apto para locomoción.
- El área debe estar libre de personas, cables, mesas, paredes y objetos bajos.
- La superficie debe ser plana y no deslizante.
- La interfaz de red debe estar confirmada con `ip link show`.
- La velocidad inicial debe ser conservadora.
- El operador debe tener acceso inmediato a una parada de emergencia física.

8.2

- No ejecutar la rutina si el robot ya está en otro modo de movimiento autónomo.
- No combinar la prueba con navegación, SLAM, teleoperación o control remoto simultáneo.
- No iniciar con distancias largas ni velocidades altas.
- No modificar los signos de `vx` o `vy` sin una prueba corta previa.
- No usar `Ctrl+C` como salida normal, porque puede interrumpir el programa antes de ejecutar `StopMove()`.

8.3

La salida normal del programa ocurre al completar los cuatro lados y ejecutar:

Listing 21: Orden normal de detención

```
client.StopMove();
```

Si el robot presenta comportamiento inesperado, se debe priorizar la parada física o el método de seguridad definido por el fabricante. Luego se debe revisar la configuración antes de volver a ejecutar.

8.4

El código actual no implementa un manejador de señales. Si se interrumpe el proceso con `Ctrl+C`, no está garantizado que `StopMove()` se ejecute. Para una versión de producción, se recomienda agregar manejo de SIGINT para llamar una función de cierre seguro.

Mejora recomendada. Implementar un manejador de interrupción que invoque `StopMove()` antes de terminar el proceso. Esto aumenta la seguridad cuando el operador necesita abortar la rutina desde terminal.

9. Guía de Uso Rápido

Esta guía resume el uso para operadores que ya conocen el entorno y solo necesitan ejecutar la rutina.

1. Conectar el computador al Unitree R1 EDU por Ethernet.
2. Verificar la interfaz:

```
ip link show
```

3. Editar el código si la interfaz no es `eth10`:

```
std::string network_interface = "eth10";
```

4. Compilar:

```
cd r1_loco_routines
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

5. Ejecutar:

```
./r1_square_mode
```

6. Ingresar una distancia y velocidad conservadoras.
7. Verificar que el robot ejecute los cuatro lados.
8. Confirmar que al final aparezca el mensaje de detención y se ejecute `StopMove()`.

10. Problemas Conocidos y Soluciones

Problema	Causa probable	Solución
No compila por cabecera no encontrada	Las rutas del SDK Unitree no están incluidas en el proyecto.	Ajustar <code>target_include_directories</code> o compilar dentro del árbol de ejemplos del SDK.

Problema	Causa probable	Solución
Error de enlazado	Falta enlazar la librería correspondiente del SDK.	Revisar el <code>CMakeLists.txt</code> usado por ejemplos oficiales del R1 y replicar las librerías necesarias.
El programa no se comunica con el robot	<code>network_interface</code> no corresponde a la interfaz real.	Ejecutar <code>ip link show</code> , identificar la interfaz Ethernet correcta y recompilar.
La interfaz aparece inactiva	Cable desconectado, interfaz abajo o configuración de red incorrecta.	Revisar cableado, ejecutar <code>ip addr</code> , activar interfaz si aplica.
El robot no se mueve	Cliente no inicializado, robot en modo no apto o comando no aceptado.	Verificar estado del robot, ejecutar primero una prueba mínima y revisar mensajes del SDK.
Movimiento lateral incorrecto	Convención de eje Y distinta o modo de locomoción limitado.	Realizar prueba corta con <code>vy = +0.1</code> y confirmar dirección real.
El cuadrado queda deformado	Rutina de lazo abierto sin odometría ni corrección de yaw.	Reducir velocidad, usar superficie plana o implementar control con realimentación.
Movimiento brusco entre lados	Cambio directo entre velocidades perpendiculares u opuestas.	Agregar <code>StopMove()</code> y pausa corta entre lados.
El robot se desvía de orientación	Pequeña deriva de yaw durante la locomoción.	Implementar corrección de orientación o reducir duración/velocidad.
Valores de entrada inválidos	Distancia o velocidad menor o igual a cero.	Ingresa valores positivos. El programa ya bloquea estos casos.
El proceso se interrumpe con <code>Ctrl+C</code>	No existe manejador de señales.	Usar parada física si es necesario y agregar manejo de <code>SIGINT</code> en una versión posterior.

11. Extensión y Mantenimiento

La herramienta puede ampliarse sin alterar su principio operativo. Las modificaciones deben mantener la secuencia lógica: validar entrada, calcular duración o parámetros, activar locomoción, ejecutar comandos y cerrar con `StopMove()`.

11.1

Para aumentar estabilidad, puede insertarse una detención corta entre movimientos:

Listing 22: Pausa segura entre lados

```
client.StopMove();  
std::this_thread::sleep_for(std::chrono::milliseconds(300));
```

Esta modificación reduce transiciones bruscas y facilita observar cada segmento por separado.

11.2

La versión actual no rota el robot. Para una rutina tipo “avanzar y girar 90 grados”, se debe usar velocidad angular `wz` o una API específica de orientación si el SDK la proporciona. La extensión debe documentar claramente la unidad usada para `wz` y validar el sentido de giro antes de ejecutar distancias largas.

11.3

En lugar de dejar `eth10` fijo, se puede recibir la interfaz como argumento:

Listing 23: Extensión sugerida: interfaz por argumento

```
int main(int argc, char** argv) {  
    std::string network_interface = "eth10";  
    if (argc > 1) {  
        network_interface = argv[1];  
    }  
  
    unitree::robot::ChannelFactory::Instance()->Init(0, network_interface);  
    // ... resto del programa  
}
```

Ejecución:

```
./r1_square_mode eth0
```

11.4

Se recomienda limitar distancia y velocidad para evitar entradas accidentales:

Listing 24: Validación adicional recomendada

```
const float MAX_DISTANCE = 2.0f;  
const float MAX_SPEED = 0.4f;  
  
if (distance > MAX_DISTANCE || speed > MAX_SPEED) {  
    std::cerr << "Error: parámetros por encima del límite seguro." << std::endl;  
    return 1;  
}
```

11.5

Elemento	Motivo
<code>ChannelFactory::Instance(network_interface)</code>	Define el modo y canal de comunicación con el robot. Un cambio incorrecto puede impedir control físico.
<code>client.Init()</code>	Inicializa el cliente requerido para comandos posteriores.
<code>client.Start()</code>	Activa el sistema antes de enviar velocidades.
<code>client.StopMove()</code>	Cierre operativo esencial para detener el movimiento.
Signos de v_x y v_y	Determinan la dirección real del movimiento. Deben validarse con pruebas cortas.

11.6

Después de cualquier cambio, ejecutar el siguiente protocolo:

1. Compilar sin advertencias críticas.
2. Probar con distancia menor o igual a 0.3 m.
3. Probar con velocidad menor o igual a 0.1 m/s.
4. Confirmar la dirección de cada eje.
5. Confirmar detención final.
6. Registrar el cambio en el README o en la bitácora del proyecto.



12. Resumen de Actividades

Actividad	Estado	Observación
Desarrollo del script de rutina cuadrada	Completado	El código implementa entrada por terminal, cálculo temporal, secuencia de cuatro lados y detención final.
Inicialización con LocoClient	Completado	Se usa ChannelFactory, client.Init() y SetTimeout.
Validación de parámetros	Completado	Bloquea distancia y velocidad menores o iguales a cero.
Secuencia translacional +X/+Y/-X/-Y	Completado	La rutina conserva $wz = 0$; no ejecuta giros.
Validación en robot físico	Pendiente de registro	Debe documentarse resultado real, interfaz usada y condiciones de prueba.
Validación de seguridad	Parcial	El código incluye StopMove() final, pero no maneja Ctrl+C.
Documentación técnica	Completado	Este documento permite replicar, ejecutar, depurar y extender la implementación.
Mejoras recomendadas	Pendiente	Agregar límites máximos, pausa explícita con StopMove() entre lados y manejo de señales.

13. Conclusión Técnica

La rutina documentada constituye una base funcional para ejecutar una trayectoria cuadrada en el Unitree R1 EDU mediante comandos de velocidad. Su principal ventaja es la simplicidad: el operador define distancia y velocidad, y el programa transforma esos valores en una secuencia de locomoción reproducible. Su principal limitación es que opera en lazo abierto, por lo que no garantiza precisión geométrica absoluta ni corrección de orientación.

Para pruebas internas, la versión actual es adecuada como rutina básica de validación de comunicación, locomoción y respuesta del robot. Para uso más robusto, se recomienda añadir límites máximos de seguridad, detención intermedia entre lados, manejo de interrupciones y, si el proyecto lo requiere, realimentación por odometría o estimación de pose.